

PRO-LONG: Programmatic Memory Enables Long-Horizon Reasoning

Alexis Fox¹, Junlin Wang¹, Paul Rosu¹, Bhuwan Dhingra¹

¹Duke University

alexis.fox@duke.edu, bdhingra@cs.duke.edu

Long-horizon tasks require sustained perception, reasoning, and exploration, and are a persistent challenge for large language model (LLM) agents. This gap is reflected in their limited performance on continual learning benchmarks such as ARC-AGI-3, especially when models are evaluated out of the box. Various agent harnesses have been proposed to close this gap, and each commits to a strategy for handling long sequences of observations, i.e., what information to save from the environment and how to load it into model context, a choice we argue is particularly consequential. Existing methods for context management face a significant tradeoff, as preserving more information makes retrieving relevant details less tractable. We propose **PRO-LONG**, a minimal context management framework built around *programmatic memory* for LLM agents in long-horizon, exploratory settings. PRO-LONG addresses the tradeoff by keeping a complete, structured interaction log and capitalizing on recent progress in coding agents to search this history efficiently. On the full ARC-AGI-3 public game set, PRO-LONG improves over a base coding agent by an average of 18.0 percentage points across frontier models, and matches or exceeds state-of-the-art specialized harnesses (up to 76.1% pass@1) while using 4.2–5.8× fewer tokens. With Table 5, PRO-LONG achieves 97.4% best@2 at a total cost of \$1,750. Relevant code and logs are available at <https://github.com/alexisfox7/PRO-LONG>.

1. Introduction

Large-language model (LLM) agents are being increasingly deployed on more challenging, longer-horizon tasks. Continual learning benchmarks such as ARC-AGI-3 [Chollet et al., 2026a] and NetHack [Küttler et al., 2020] have subsequently been proposed to evaluate agentic progress, testing whether these agents can infer unknown environment dynamics through exploration and continuously improve over thousands of actions. Out-of-the-box frontier LLMs, evaluated without harnesses or tool access, perform poorly on these settings, achieving near-zero scores on ARC-AGI-3 and limited progression on NetHack [Paglieri et al., 2025].

To close this gap, a range of agent harnesses has been proposed for exploratory tasks, wrapping a base model with memory, tools, and other scaffolding, which have shown considerable promise. In parallel, and of particular relevance, agents with access to programmatic tools, such as coding agents, have advanced rapidly. Programmatic access enables forms of memory and reasoning that were previously impractical, such as structured search over very large logs. Recent work externalizes long-context processing in a similar spirit [Cao et al., 2026], for example through recursive language models that query slices of context through a REPL [Zhang et al., 2025]. These trends point toward a broader role for coding agents as context processing systems.

For these agents to maintain long-horizon competence and coherence, how to properly engineer context becomes a central concern. A natural way to frame agent context management is to distinguish between the *accessed* information, through the active context window, and the *accessible* information, namely memory (i.e., tool-reachable state such as a log file, saved notes, or an embedding store). Although modern LLMs can process context windows of near to a million tokens, performance remains expensive and vulnerable to degradation as trajectories grow, i.e., context rot [Hong et al., 2025]. Agents therefore require mechanisms that decide what history remains accessible and how relevant pieces are moved into context.

Proposed agent memory systems can be understood as various implementations of `READ` and `WRITE` operations

over externally stored information [Zhang et al., 2024]. `WRITE` is defined by heuristics for what information should be accessible or persisted, whether agent-authored, human-designed, learned, or task-specific. Examples include scratchpad notes, summarized observations, embedded memory entries, and distilled skills or strategies [Park et al., 2023, Packer et al., 2023, Chhikara et al., 2025, Ouyang et al., 2025, Xia et al., 2026]. `READ` then determines what is ultimately accessed in context through analogous operations, such as embedding-based retrieval, loading selected memory chunks, keyword search over stored files, or restoring earlier context snapshots [Lewis et al., 2020, Wu et al., 2025]. This creates a fidelity–tractability tradeoff. Reducing what is saved at `WRITE` time makes `READ` easier, but can discard details whose relevance is evident only in hindsight. Over long horizons, the cost of these decisions accumulates, as a lossy summary can matter many turns later. A timeliness question is also raised as to whether these systems are fully capitalizing on, or are maximally compatible with, modern coding agent capabilities.

We propose **PRO-LONG**, a context management framework for eliciting **programmatic, long-horizon** reasoning. PRO-LONG centers on our notion of programmatic memory for agents, which we define the `WRITE` operation as the harness appending every observation, action, and outcome into a log, and the `READ` as programmatic search over that log, primarily through code. We emphasize three motivating design principles. First, *simplicity*; the `WRITE` needs no learned or heuristic decisions about what information to store, since everything is added as the environment outputs it. `READ` requires no specialized retrieval, e.g., accessing a vector database or index. Second, *losslessness*; nothing is compressed or summarized when writing, so the log is a faithful, ground-truth record of environment state. Third, the approach is *uniquely enabled by, and especially compatible with, coding agents*. Programmatic search, via regular expressions or code, is native to coding agents [Cao et al., 2026], and this capability keeps retrieval tractable even over very long logs; in practice, we find PRO-LONG retrieval to be tractable for logs over 100k+ lines.

Contributions. We find that PRO-LONG achieves state-of-the-art results on ARC-AGI-3, and attribute its performance to, and organize our contributions around, the three design principles above.

- **Framework.** We introduce PRO-LONG, a context framework optimized for long-horizon coherence, which introduces programmatic memory for agents as an append-all `WRITE` to a log and a code-based `READ` over that log. Our design is motivated by simplicity, losslessness of information, and maximal compatibility with coding agent capabilities.
- **ARC-AGI-3 Performance.** Across model families and under matched scoring setups, PRO-LONG achieves performance comparably to or better than specialized harnesses while using $4.2\text{--}5.8\times$ fewer billed tokens (§3). With Opus 4.6, it scores 42.4% pass@1, the best public result for that model. With GPT-5.5, PRO-LONG reaches 41.2% pass@1 and 60.1% best@5, compared with the prior best of 45.1%, while using $5.8\times$ fewer tokens. With Fable 5 at a full action budget, we obtain 97.4% best@2 at less than one-third the cost of the strongest prior harness. Despite being a minimal addition to standard coding agents (Codex and Claude Code), PRO-LONG improves over the same agents without the log by 18.0 percentage points on average.
- **Drivers of Performance.** We show that adding PRO-LONG to base coding agents with even minimal tooling performs strongly, supporting the simplicity principle, and that the gains are driven by full log access. Performance improves steadily with increasingly powerful programmatic access (Table 1), while further abstractions, such as persistent workspaces and tools for writing notes, add little (Table 2). Improvements are clearest on games where long-horizon reasoning is most useful, i.e., the current board does not fully determine the game dynamics (Figure 4).

PRO-LONG Context Framework

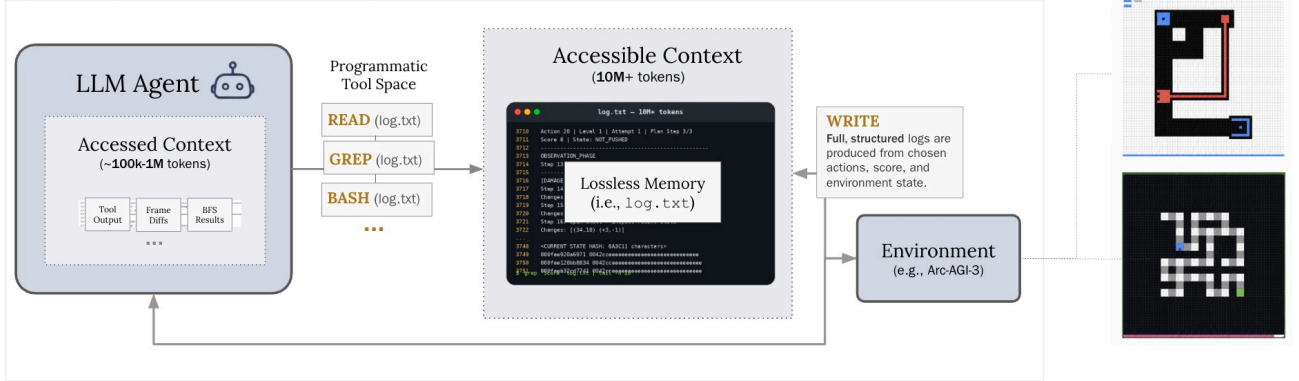


Figure 1: Overview of programmatic memory (PRO-LONG) integrated into a basic coding agent setup. The agent has access to a tool space of increasingly programmatic capabilities (read, grep, bash/python), and full, structured logs of environment state are written to `log.txt`. Note the accessed context (limited to ~100k–1M tokens) versus the far larger accessible context (10M+ tokens). Two example ARC-AGI-3 games are shown on the right: `g50t`, which requires controlling a “ghost clone,” and `tu93`, a maze game.

2. Setup & Methods

2.1. Environment

We evaluate on the ARC-AGI-3 benchmark; each ARC-AGI-3 environment consists of six to ten levels of increasing difficulty, with 25 public games in total. The rules of these games are not disclosed. Thus, the agent must infer environment dynamics through exploration. We detail specific environment details and evaluation protocol below.

Environment representation. To provide intuition for the games in the benchmark, Figure 1 shows example board states from two: `tu93`, a maze game in which the agent must reach a goal tile while disarming turrets, and `g50t`, where “rewinding” creates a ghost that repeats the agent’s previous path, so runs must be planned such that ghosts hold switches open. For all games, boards are 64×64 grids of pixels, each taking one of 16 colors; we describe how boards are presented to the agent in the evaluation protocol below. The action space consists of four directional moves, a click targeting a specific (x, y) cell, undo, and reset. Available actions vary by game, from click-only to the full set, and most games have a per-level action budget or timer that forces a reset when exceeded.

Scoring. Our scoring follows the benchmark standard, *Relative Human Action Efficiency* (RHAЕ) [Chollet et al., 2026a]. The per-level score compares the agent’s action count $a_{l,e}$ against an upper-median human baseline $h_{l,e}$ as $S_{l,e} = \min((h_{l,e}/a_{l,e})^2, 1.15)$, so an agent that uses fewer actions than the human baseline earns up to 115% per level. Per-environment scores weight later levels more heavily and are capped at the weighted fraction of levels completed, and the benchmark score T averages over all $|D|$ environments:

$$T = \frac{1}{|D|} \sum_{e \in D} \frac{\sum_{l=1}^n l S_{l,e}}{n(n+1)/2}. \quad (1)$$

Evaluation protocol. Unless otherwise specified, our default configuration for all evaluated agents sets high model reasoning effort and a maximum of 500 actions per game, with at most 20 actions per turn. For all agents that do not enable vision, including ours, boards are represented as 64×64 text grids, with one character per pixel and with 16 colors mapped to ASCII symbols (color map in Appendix B). Unless otherwise stated, GPT models

use the Codex agent framework and Claude models use Claude Code, with their standard tool space (e.g., `read`, `write`, `bash`, `grep`) in a sandboxed container that provides `python3`. Our agents have access only to the Python standard library.

We see significant run-to-run variation for most models on ARC-AGI-3, as discussed further in §3. We therefore report both `pass@1` and `best@k` when compute permits. Each repeated run is independent, with a fresh session and workspace for each game, so no information is transferred across runs or games. We compute `pass@1` by averaging replicate scores within each game and then averaging across the fixed set of 25 games. For `best@k`, we take the maximum score for each game over k runs, average over all k -subsets of the available replicates, and then average across games. We calculate 95% bootstrap confidence intervals by resampling runs within each game. All agents are evaluated using the same 500 action budget and scoring procedure. Because prior agents were originally evaluated under different action budgets or scoring versions, we rescore their publicly available runs to ensure a fair comparison.

2.2. PRO-LONG & Agent Architectures

We build on coding agents, i.e., LLMs operated in a loop with access to a file system and a shell, able to read and write files and execute code as part of their reasoning. Following §1, we distinguish the agent’s *accessed* state, the information in its active context window at a model call (e.g., system prompt, the current board, and recent tool outputs), from its *accessible* state, the external memory it can reach only through tools. Figure 1 visualizes how our context framework, PRO-LONG, integrates into a basic coding agent. Below, we define each agent architecture evaluated, including other public baselines. Full prompts for our agents are provided in Appendix B.

Base coding agent. We define a simple coding agent baseline to measure the effectiveness of adding PRO-LONG to a minimal setup. At each turn, the agent reasons over its accessed context of up to $\sim 250k$ tokens, may call the tools described in §2.1, and outputs a short sequence of actions. The environment executes these actions and then calls the agent again, with the resulting board added to the user prompt. Earlier boards may remain in the active context window, but the baseline provides no external memory beyond what the agent writes itself, e.g., notes or saved boards in workspace files; in practice, it relies heavily on such workspace writing (Table 3). We denote this baseline *no-log* and use it as our primary point of comparison. This baseline is stronger than the standard agents in continual learning benchmarks, which typically are only given the current state and a short history, without coding tools. For example, ARC-AGI-3 reference agents are given the last 3–5 board states through a 10-message window [ARC Prize Foundation, 2026], and BALROG agents receive the last 16 observations [Paglieri et al., 2025].

PRO-LONG. We add PRO-LONG, our minimal context framework, to the base coding agent above. Figure 1 shows the corresponding `WRITE` and `READ` operations for this harness. For each action, the `WRITE` appends a structured entry `logs.txt` containing a header (the action number, level, attempt, and score), the agent’s summarized plan, the selected action, and the resulting board. The `READ` searches over this log with the programmatic tools, i.e., regular expressions or code, of effect we discuss in §3.2. With these tools, the agent can, for instance, `grep` the log for score changes to identify the most consequential actions for a level. An interesting example we see is a PRO-LONG agent that writes a `regress.py` script to replay every action in `log.txt` and verify its coded game model predicts the logged board states. We go over more examples on specific games in §3.3. We also study an intermediate condition for PRO-LONG that truncates the log to the last 25 actions (§3.1). We use no subagents, and our prompt is only around 30 lines (Appendix B.1).

Baselines. In addition to our base coding agent, *no-log*, we compare against WorldModeler¹ [Rodionov, 2026] (GPT-5.5), Arcgentica [Symbolica AI, 2026] (Opus 4.6), and the Schema harness [Impossible Research, 2026] (Opus 4.8 and Fable 5). We focus on these three as the strongest public harnesses for the frontier models evaluated, that

¹The released agent of Rodionov [2026] is named `baseline1`; we refer to it as WorldModeler for clarity, given the executable world model emphasis in their implementation.

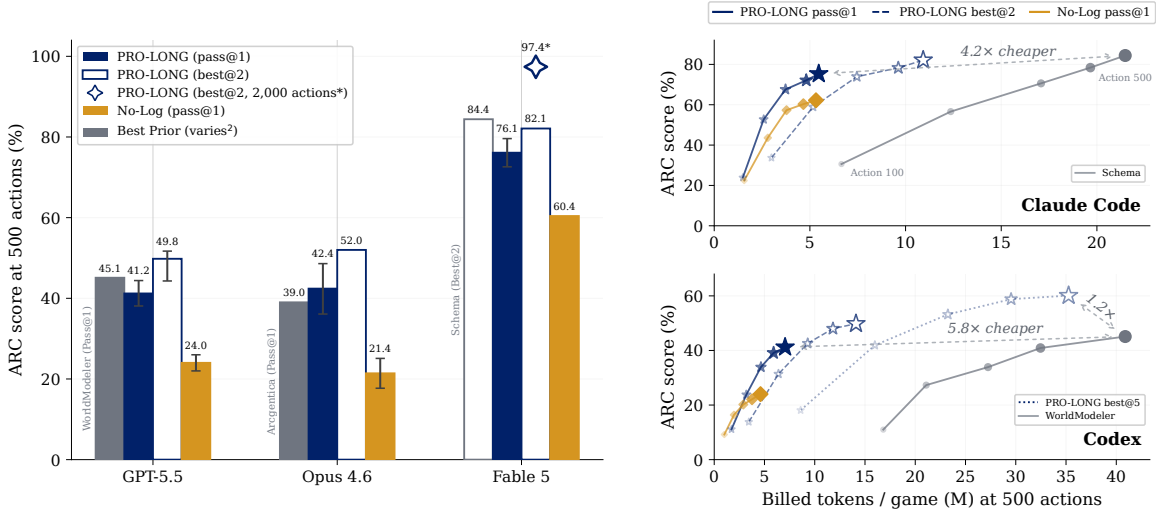


Figure 2: PRO-LONG matches or exceeds state-of-the-art ARC-AGI-3 results at 4.2–5.8× lower token cost. *Left:* ARC-AGI-3 results on the public game set with a 500 action limit, grouped by model and harness. Filled bars show pass@1, with bootstrap confidence intervals where multiple runs are available (five for Codex, two for Claude Code). Outlined bars show best@ k . We rescore the released runs of WorldModeler, Arcgentica, and Schema for a consistent comparison.² Schema reports best@2; the others report pass@1. *Right:* Performance versus billed tokens per game for PRO-LONG and the strongest prior harness on Codex and Claude Code, across budgets from 100 to 500 actions. PRO-LONG stays within 2–4 points of the strongest prior harness at 4.2–5.8× lower cost.

*PRO-LONG (Fable 5) at a 2,000-action limit; this is a lower bound on best@2, as certain games we only ran once.

also follow the standard assumptions of ARC’s official leaderboard, including independent runs with no information shared across games.

WorldModeler and Schema both focus on having the agent create coded “world models” of the games. WorldModeler, a harness for Codex, has a prompt of around 600 lines instructing the agent to write and continually update a Python simulator of the game. The harness provides helper scripts and defines two subagents for tasks like frame preprocessing and animation analysis. WorldModeler is also the only agent evaluated that uses vision. Schema follows a similar approach on Claude Code, replacing the default tools with twelve custom ones (through an MCP server). Across turns, the Schema agent updates a `step(grid, action)` function predicting the next board, calls a backtest tool to validate the model, and searches for plans by breadth-first search. Arcgentica is built on a custom agent SDK, and defines a main orchestrator agent that delegates to four subagents (Explorer, Theorist, Tester, and Solver), which all update a shared memory database of hypotheses and findings.

3. Main Results

We evaluate PRO-LONG on the full ARC-AGI-3 public game set. Despite being a minimal addition to a basic coding agent, PRO-LONG improves significantly over the no-log baseline, and matches or outperforms specialized harnesses with much lower token costs. We present benchmark performance (§3.1), ablate which components drive it (§3.2), and describe a qualitative analysis of agent behavior (§3.3).

3.1. Benchmark Performance

Figure 2 compares benchmark performance for PRO-LONG and the relevant baselines discussed in §2.2. Across all frontier models tested, namely GPT 5.5, Opus 4.6, and Fable 5, PRO-LONG consistently improves by 15.7–21.0

percentage points over the base coding agent baseline (no-log, pass@1). We provide further intuition for this result, and how PRO-LONG incentivizes long-horizon reasoning, in §3.2 and §3.3.

Under matched scoring setups,² PRO-LONG either surpasses specialized harnesses (Opus 4.6, +3pp at pass@1) or is within comparable range (2–4pp on GPT-5.5 and Fable 5). We emphasize that PRO-LONG’s simple design is model-agnostic in implementation, whereas prior harnesses are typically tuned to a specific model. This simplicity also translates into substantial token savings.

In Figure 2, we also compare the billed token cost of our method against the best prior harness on each coding agent, Claude Code and Codex. On Codex, PRO-LONG reaches 41.2 pass@1 versus WorldModeler’s 45.1 while using $5.8\times$ fewer tokens. At best@5, it reaches 60.1 while remaining $1.2\times$ cheaper. On Claude Code, PRO-LONG reaches 82.1 best@2, close to Schema’s 84.4 best@2 while using $4.2\times$ fewer tokens. Across the full game set, this corresponds to savings of roughly **400 million tokens**.

Beyond the matched 500-action comparison, we also run Fable 5 at a 2,000-action budget, where PRO-LONG scores 94.6 pass@1 at a cost of \$1,500 (150M billed tokens). PRO-LONG reaches **97.4%** best@2 at a total cost of \$1,750; this is a lower bound on best@2, as several games were not rerun. A few difficult games account for most of the cost; bp35 alone costs \$298. Schema releases logs only for its retained best runs (up to 3,000 actions),³ whose reported cost totals \$6,447 for 99.0 best@2; even this partial total is more than $3\times$ higher than ours.

In Figure 3, we do further best@ k analysis, looking at how score improves as k increases and how variants of PRO-LONG compare. Note, first, there is significant variance in game outcomes across independent runs, and thus it is important to report best@ k results where possible. For example, GPT-5.5 reaches 41.2 at pass@1, while best@5 rises by 18.9 percentage points to 60.1. Second, increasing the “fullness” of memory leads to a large improvement in score, and this gap widens with k . The widening gap suggests that fuller memory covers more games, i.e., expands the set of games solvable at all, and that games requiring long-horizon reasoning are harder in general, with higher variance that may reflect interesting exploration failure modes.

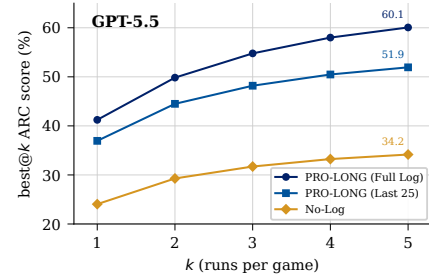


Figure 3: Best@ k comparison (GPT-5.5) for PRO-LONG, a truncated version (log keeps only the last 25 actions), and no-log. The gap widens with k .

3.2. General Ablations

Note that since PRO-LONG builds on a base coding agent, it can WRITE other forms of memory beyond the log, such as self-authored notes. A core property of how useful such memory is relies on whether it is persistent, i.e., notes must be maintained across model calls so the agent can refer back to them. We ablate how much PRO-LONG is affected by disabling the persistence of these other forms of accessible context, through wiping the workspace every model call. We find that for the no-log agent, which relies heavily on self-authored notes and helper functions (Table 3), performance degrades by roughly four points, while PRO-LONG is essentially unaffected, dropping only 0.5 points (Table 2).

3.3. Qualitative Analysis

We show individual game scores statistics over five runs for PRO-LONG and our base coding agent in Figure 4, which helps explain where programmatic memory provides the greatest benefit. Both the baseline and PRO-LONG perform strongly on some games, such as ft09 and cd82. In these cases, the current board largely specifies the

²For prior agents, the selection procedure is not always public and replicates are not available. WorldModeler evaluates a single playthrough per game, and Arcgentica releases one run per game with no stated selection procedure; we label both pass@1. Schema uses a best-of-two heuristic over Opus 4.8 and Fable 5, which we compare against our best@2.

³Since artifacts exist only for retained runs, cost comparisons for Claude use retained run costs. Even comparing the full cost of our best@2 (both runs) against Schema’s single retained runs, PRO-LONG remains $2.0\times$ cheaper. For Codex, we instead report the full best@5 cost (all five runs) to emphasize the savings relative to WorldModeler’s pass@1.

Tools enabled	Score (%)
Read only	23.1
+ Grep / Regex Search	27.2
+ Python	38.3
+ Write, Edit	41.2

Table 1: Tool ladder ablation along the hierarchy of programmatic access (GPT 5.5). Increasing level of programmatic control over logs and programmatic reasoning lead to improved performance, whereas other types of tooling such as write do not.

Workspace	PRO-LONG	No-Log
Persistent	41.2 $\pm$ 3.5	24.0 $\pm$ 2.0
Cleared every call	40.7 $\pm$ 3.6	19.9 $\pm$ 2.1

Table 2: Workspace persistence ablation. pass@1 (%) for GPT 5.5 with bootstrap confidence intervals ($n = 5$ persistent, $n = 2$ cleared). Clearing the workspace after every call has little effect on PRO-LONG but reduces no-log performance by about four points.

puzzle dynamics, and thus, any access to the full trajectory provides little additional information. For example, `ft09` requires recoloring boxes to match a pattern that is usually visible directly on the current board; most agents we evaluate complete this game in 100–200 actions.

Programmatic memory is most useful for games that require longer-horizon reasoning, like `m0r0` and `g50t`. For `g50t` (Figure 1), individual PRO-LONG run logs exceeded 320,000 lines. The primary game mechanic is “rewinding,” which creates ghosts that replay the agent’s previous paths; later levels require coordinating multiple ghosts as they move and teleport toward switches. Even with a lengthy log and complex game dynamics, PRO-LONG scores up to 56.3% with GPT 5.5. For `m0r0`, the agent controls two blocks in a maze-like game. Although our setup does not require or emphasize “world models,” the PRO-LONG agent codes one on its own, defining transition functions that grow with later levels (e.g., by adding a switch state variable) and running breadth-first search over joint block positions to find valid routes of more than 50 actions. PRO-LONG scores 100% on all five runs, compared with 34.2% for the no-log baseline.

The tool use statistics across the full set of games, in Table 3, support a similar attribution of PRO-LONG’s gains to differences in analysis style. PRO-LONG uses most of its calls on programmatic tools, with `python` accounting for 60.6% of calls, compared with 40.8% for no-log, and regular expression search over the log, such as `grep`, accounts for 9.6%. No-log instead uses most of the calls on workspace management and writing other forms of memory, like notes, with workspace writing calls increasing from 4.6% (PRO-LONG) to 30.3%. The conclusion is similar to that of Table 1, which is that other forms of memory, such as those from writing, do not seem essential for performance.

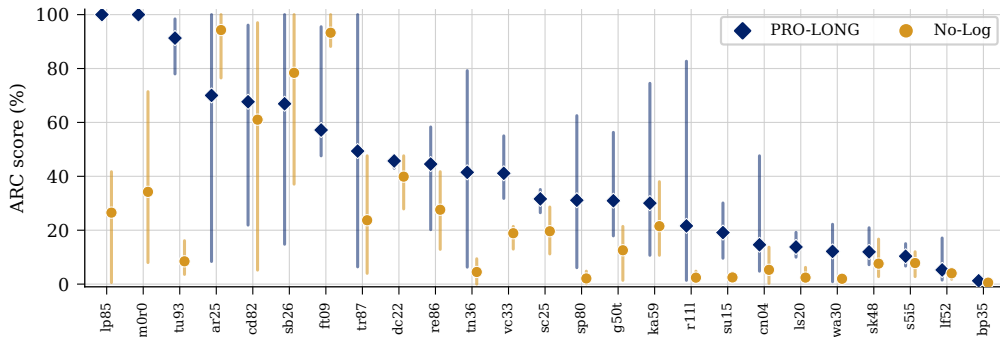


Figure 4: Score breakdown across the 25 games, PRO-LONG vs. no-log for GPT 5.5 runs, sorted by PRO-LONG mean. Markers show the pass@1 average per game and bars span the minimum and maximum score over $n = 5$ runs. PRO-LONG matches or exceeds no-log scores on nearly every game.

Category	Tools	PRO-LONG	No-Log
Programmatic analysis	Python3	60.6%	40.8%
Log parsing	Tools used on logs.txt	20.3%	0.0%
Recent state	Tail, head, cat	10.7%	–
Search across turns	Grep, sed, awk, wc, find, ls	9.6%	–
Workspace management		19.1%	59.2%
Navigation	Find, ls, pwd	14.6%	29.0%
Writing	Apply_patch, cat, tail on notes, plans	4.6%	30.3%

Table 3: PRO-LONG vs. no-log tool call distribution for GPT 5.5 runs. Across the 25 games, PRO-LONG spends more time on programmatic analysis (40.8% $\rightarrow$ 60.6%); no-log makes more workspace calls (19.1% $\rightarrow$ 59.2%). Action output writing is excluded for both agents to focus the comparison on reasoning differences.

4. Related Works

4.1. Context Engineering & Management

LLMs have finite context windows, and their performance often declines as context gets longer due to context rot [Liu et al., 2023, Hong et al., 2025]. This limitation is particularly important for long-horizon tasks that involve substantial exploration, because the history of observations, actions, and intermediate results can eventually exceed the available context. Recent work on context engineering addresses this problem by storing information outside the model’s active context and retrieving it when needed [Mei et al., 2025]. Externalized-memory systems can be defined by how they write, read, and synthesize information. Nye et al. [2021] let the agent write summaries into a scratchpad. MemGPT [Packer et al., 2023] uses a tiered memory system that moves information in and out of a fixed context window. Other systems, including Mem0 [Chhikara et al., 2025] and ReasoningBank [Ouyang et al., 2025], turn past experience into entries that can later be retrieved. Skill-library methods such as SkillRL [Xia et al., 2026] convert trajectories into reusable skills. These methods reduce the amount of information kept in the active context, but they must decide in advance what to store or summarize. This can be difficult because information that appears unimportant at one point may become useful later.

Another approach is to preserve the original information and decide what is relevant only when it is needed. Recursive language models [Zhang et al., 2025], for example, use code to inspect or query parts of a long input through a REPL, and coding agents have also been used for broader long-context processing [Cao et al., 2026]. PRO-LONG follows this approach by keeping a complete append-only log of the agent’s trajectory. The log does not need to be summarized or reorganized as the task proceeds. Instead, the agent can use code to search the log and retrieve relevant parts during later reasoning.

4.2. ARC-AGI-3

ARC-AGI-3 [Chollet et al., 2026a] extends ARC-AGI-2 [Chollet et al., 2026b] and the original Abstraction and Reasoning Corpus [Chollet, 2019] from static input-output puzzles to interactive games. In these games, an agent must infer the rules by taking actions and observing their effects. The benchmark therefore evaluates abilities that are difficult to measure with static tasks alone, including exploration, goal inference, and planning over long sequences of actions. ARC-AGI-3 is related to earlier interactive game benchmarks such as the NetHack Learning Environment [Küttler et al., 2020] and BALROG [Paglieri et al., 2025]. On these benchmarks, unmodified language models also tend to make limited progress, while additional scaffolding can substantially improve performance [Vaguely Aligned, 2026].

On ARC-AGI-2’s static grid tasks, strong approaches include test-time adaptation, as used by NVIDIA’s NVARC; small task-specific networks trained from scratch [Jolicoeur-Martineau, 2025, Wang et al., 2025, Liao and Gu, 2025]; and LLM systems augmented with external tools or memory [Pourcel et al., 2025, Ho et al., 2025]. ARC-AGI-3

presents a different challenge because agents must learn the rules of each game through interaction. In the preview results, unscaffolded LLMs scored below 1%, while several non-LLM methods performed substantially better. StochasticGoose [Smit, 2025], which combines a convolutional neural network with sparse-reward reinforcement learning, achieved 12.58%, while Blind Squirrel, which explores the environment by constructing directed graphs of observed states, achieved 6.71%.

5. Conclusion

We introduce **programmatic memory** for **long**-horizon reasoning with our framework PRO-LONG. We claim that (1) programmatic perception and reasoning, such as using code to parse board grids, model game transitions, and plan with breadth-first search, are broadly effective for continual learning tasks, and (2) simple harnesses can elicit these behaviors, with PRO-LONG a minimal but highly effective instance. We emphasize simplicity, alongside our other design principles of lossless memory and compatibility with coding agents. Even with a minimal prompt and agent architecture, PRO-LONG frequently creates sophisticated models of game dynamics and long-horizon action plans. As such, our model-agnostic method is competitive across all evaluation settings, uses significantly fewer tokens than the strongest prior harness on each model, and achieves up to 97.4% on the public ARC-AGI-3 benchmark.

We evaluate across various frontier models, two of which were released after the benchmark. Although this may advantage agents using those models, including the baselines, ARC-AGI-3 remains useful for relative comparison of long-horizon reasoning and context management methods. We also note the significant variance we see across game runs, and thus report $\text{best}@k$ where possible. This variance motivates several directions for future work. Given the high cost of evaluating on continual learning benchmarks, especially for repeated runs, an important direction is to see if reinforcement learning and improved harness designs can close the gap between $\text{pass}@1$ and $\text{best}@k$. Other exciting directions include co-training models with the harness, and studying how programmatic memory interacts with other memory mechanisms, like scratchpad notes and compression.

6. Acknowledgments

We gratefully acknowledge the ARC Prize Foundation for providing computational resources and credits used in this work, as well as for valuable discussions and feedback. Junlin Wang is partially supported by through Grant G-23- 2137070 from the Learning Engineering Virtual Institute to the University of Florida and its partner institutions, including Duke.

References

- ARC Prize Foundation. ARC-AGI-3 reference agents. <https://github.com/arcprize/ARC-AGI-3-Agents>, 2026.
- Weili Cao, Xunjian Yin, Bhuwan Dhingra, and Shuyan Zhou. Coding agents are effective long-context processors, 2026. URL <https://arxiv.org/abs/2603.20432>.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory, 2025. URL <https://arxiv.org/abs/2504.19413>.
- François Chollet, Mike Knoop, and Greg Kamradt. ARC-AGI-3: A new challenge for frontier agentic intelligence. *arXiv preprint arXiv:2603.24621*, 2026a. URL <https://arxiv.org/abs/2603.24621>.
- François Chollet, Mike Knoop, Gregory Kamradt, Bryan Landers, and Henry Pinkard. Arc-agi-2: A new challenge for frontier ai reasoning systems, 2026b. URL <https://arxiv.org/abs/2505.11831>.
- François Chollet. On the measure of intelligence, 2019. URL <https://arxiv.org/abs/1911.01547>.
- Matthew Ho et al. ArcMemo: Abstract reasoning composition with lifelong LLM memory, 2025. URL <https://arxiv.org/abs/2509.04439>.
- Kelly Hong, Anton Troynikov, and Jeff Huber. Context rot: How increasing input tokens impacts llm performance. <https://www.trychroma.com/research/context-rot>, 2025.
- Impossible Research. Schema: Frontier models with our harness achieve ~99% on ARC-AGI-3 public. <https://schema-harness.github.io>, 2026. Released 2026-07-15.
- Alexia Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks, 2025. URL <https://arxiv.org/abs/2510.04871>.
- Heinrich Küttler, Nantas Nardelli, Alexander H. Miller, Roberta Raileanu, Marco Selvatici, Edward Grefenstette, and Tim Rocktäschel. The NetHack learning environment. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks, 2020. URL <https://arxiv.org/abs/2005.11401>.
- Isaac Liao and Albert Gu. ARC-AGI without pretraining, 2025. URL <https://arxiv.org/abs/2512.06104>.
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts, 2023. URL <https://arxiv.org/abs/2307.03172>.
- Lingrui Mei, Jiayu Yao, Yuyao Ge, Yiwei Wang, Baolong Bi, Yujun Cai, Jiazhi Liu, Mingyu Li, Zhong-Zhi Li, Duzhen Zhang, Chenlin Zhou, Jiayi Mao, Tianze Xia, Jiafeng Guo, and Shenghua Liu. A survey of context engineering for large language models, 2025. URL <https://arxiv.org/abs/2507.13334>.
- Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, Charles Sutton, and Augustus Odena. Show your work: Scratchpads for intermediate computation with language models, 2021. URL <https://arxiv.org/abs/2112.00114>.
- Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. Reasoningbank: Scaling agent self-evolving with reasoning memory, 2025. URL <https://arxiv.org/abs/2509.25140>.

- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems, 2023. URL <https://arxiv.org/abs/2310.08560>.
- Davide Paglieri, Bartłomiej Cupiał, Samuel Coward, Ulyana Piterbarg, Maciej Wolczyk, Akbir Khan, Eduardo Pignatelli, Łukasz Kuciński, Lerrel Pinto, Rob Fergus, Jakob Nicolaus Foerster, Jack Parker-Holder, and Tim Rocktäschel. BALROG: Benchmarking agentic LLM and VLM reasoning on games. In *International Conference on Learning Representations (ICLR)*, 2025.
- Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior, 2023. URL <https://arxiv.org/abs/2304.03442>.
- Julien Pourcel, Cédric Colas, and Pierre-Yves Oudeyer. Self-improving language models for evolutionary program synthesis: A case study on ARC-AGI, 2025. URL <https://arxiv.org/abs/2507.14172>.
- Sergey Rodionov. Executable world models for ARC-AGI-3 in the era of coding agents, 2026. URL <https://arxiv.org/abs/2605.05138>.
- Dries Smit. StochasticGoose: 1st place in the ARC-AGI-3 agent preview competition. <https://github.com/DriesSmit/ARC3-solution>, 2025.
- Symbolica AI. From 0% to 36% on day 1 of ARC-AGI-3. <https://www.symbolica.ai/blog/arc-agi-3>, 2026.
- Vaguely Aligned. It’s 2026. Can LLMs Play NetHack Yet? <https://kenforthewin.github.io/blog/posts/nethack-agent/>, 2026.
- Guan Wang, Jin Li, Yuhao Sun, Xing Chen, Changling Liu, Yue Wu, Meng Lu, Sen Song, and Yasin Abbasi Yadkori. Hierarchical reasoning model, 2025. URL <https://arxiv.org/abs/2506.21734>.
- Yaxiong Wu, Sheng Liang, Chen Zhang, Yichao Wang, Yongyue Zhang, Huifeng Guo, Ruiming Tang, and Yong Liu. From human memory to AI memory: A survey on memory mechanisms in the era of large language models, 2025. URL <https://arxiv.org/abs/2504.15965>.
- Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning, 2026. URL <https://arxiv.org/abs/2602.08234>.
- Alex L. Zhang, Tim Kraska, and Omar Khattab. Recursive language models, 2025. URL <https://arxiv.org/abs/2512.24601>.
- Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model based agents, 2024. URL <https://arxiv.org/abs/2404.13501>.

A. Scoring and Token Cost Details

Games are run and scored with the official `arc_agi` engine (version 0.9.7), with scores computed according to Equation 1 and game versions pinned to those listed in Table 4.

Table 4: Game versions used for all runs in our experiments.

ar25-0c556536	bp35-0a0ad940	cd82-fb555c5d	cn04-2fe56bfb	dc22-fdcac232
ft09-0d8bbf25	g50t-5849a774	ka59-38d34dbb	lf52-271a04aa	lp85-305b61c3
ls20-9607627b	m0r0-492f87ba	r111-495a7899	re86-8af5384d	s5i5-18d95033
sb26-7fbdac44	sc25-635fd71a	sk48-d8078629	sp80-589a99af	su15-1944f8ab
tn36-ef4dde99	tr87-cd924810	tu93-0768757b	vc33-5430563c	wa30-ee6fef47

We report cost in billed tokens. Each token class is weighted by its list price relative to one uncached input token of a reference model. Multiplying a run’s billed-token count by the reference input price therefore recovers its dollar cost. Token counts are taken from the usage reported by each API call and summed over the run.

Claude models (reference: Table 5 input, \$10/M):

$$\text{billed} = \text{input} + 5 \text{ output} + 0.1 \text{ cache_read} + 2 \text{ cache_write}, \quad (2)$$

where `input_tokens` already excludes cached tokens. Cache writes use the one-hour rate because Claude Code issues only one-hour-TTL writes; we verified that `ephemeral_5m` = 0 across our runs. Opus has the same relative prices, so the same weights apply.

Codex models (reference: GPT-5.5 input, \$5/M):

$$\text{billed} = (\text{input} - \text{cached}) + 6 \text{ output} + 0.1 \text{ cached}, \quad (3)$$

No cache-write term is included because GPT-5.5 does not charge separately for cache writes.

All token comparisons in the paper use the same model family, so relative billed-token counts do not depend on the chosen reference model.

B. Full Prompts

B.1. System Prompt: (PRO-LONG)

System Prompt (Full Log)

```
You are a coding agent playing a grid-based puzzle game by writing Python action plans.
Your primary objective is to solve all levels in the game. Your secondary objective is to minimize
total cumulative actions used.

'./logs.txt' is the game log: action headers, tool calls, board states, and your own prior analyses.
log_window_desc Parse it programmatically, as reading full 64x64 board states from prompt can
introduce precision errors.cross_turn_hint

**Tools**: Read, Write, Edit, Bash, Grep, Glob.

**Workspace**: './' persists across calls. 'actions.json' is cleared each call; other files
accumulate. Feel free to save notes, state, or helper functions.

**Log markers**: [INITIAL BOARD STATE] -- the grid at the start (after Action 0 header)
[POST-ACTION BOARD STATE] -- the grid after each action [frame 1/N] ... [settled] -- animation
frames; the grid following [settled] is the committed state

**Game structure and strategy**: - Score increase means that a level was cleared. - Most games
have a step budget or timer mechanism, which will cause a level reset if exceeded. - For parsing
the boards, programmatic options include identifying connected components (color, position, size,
shape) and forming testable hypotheses about what each represents (player, walls, goals, UI, etc.).
```

```

**Response format**: a strategic briefing, then [PLAN] <2-3 sentence action plan>
**Write './actions.json'** with a JSON object {"actions": ["ACTION6(30,40)", "ACTION1", "RESET"]}
-- a list of 1-action_cap actions to execute in order. Entries beyond action_cap are discarded.
Prefer short lists (1-2 actions) when testing a new hypothesis so you see the result before
committing further; scale up toward action_cap for proven sequences.
**Actions available in this game**: actions_section
The runner executes the list in order, then calls you again with the updated log.

```

Color map appended to system prompt:

Color Map (Hex Mode)

```

**Color Map (hex digit -> color):** COLOR_MAP = { '0': 'White', '1': 'Off-White', '2': 'Light
Gray', '3': 'Gray', '4': 'Off-Black', '5': 'Black', '6': 'Magenta', '7': 'Light Magenta',
'8': 'Red', '9': 'Blue', 'a': 'Light Blue', 'b': 'Yellow', 'c': 'Orange', 'd': 'Maroon', 'e':
'Green', 'f': 'Purple', }

```

B.2. System Prompt: No-Log

System Prompt (No-Log)

```

You are a coding agent playing a grid-based puzzle game by writing Python action plans.
Your primary objective is to solve all levels in the game. Your secondary objective is to minimize
total cumulative actions used.

The current 64x64 board is injected directly into your user prompt each turn. Prior board states
and analyses are not preserved across calls unless you save them yourself to the workspace; if you
need programmatic analysis, Write the board text to a file first.

**Tools**: Read, Write, Edit, Bash, Grep, Glob.
**Workspace**: './' persists across calls. 'actions.json' is cleared each call; other files
accumulate. Feel free to save notes, state, or helper functions.
**Prompt markers** (on the user turn): [CURRENT BOARD STATE] -- the live 64x64 grid
**Game structure and strategy**: - Score increase means that a level was cleared. - Most games
have a step budget or timer mechanism, which will cause a level reset if exceeded. - For parsing
the boards, programmatic options include identifying connected components (color, position, size,
shape) and forming testable hypotheses about what each represents (player, walls, goals, UI, etc.).
**Response format**: a strategic briefing, then [PLAN] <2-3 sentence action plan>
**Write './actions.json'** with a JSON object {"actions": ["ACTION6(30,40)", "ACTION1", "RESET"]}
-- a list of 1-action_cap actions to execute in order. Entries beyond action_cap are discarded.
Prefer short lists (1-2 actions) when testing a new hypothesis so you see the result before
committing further; scale up toward action_cap for proven sequences.
**Actions available in this game**: actions_section
The runner executes the list in order, then calls you again with an updated board.

```

B.3. User Prompts

B.3.1. PRO-LONG

First Call

```

Read the full game log at ./logs.txt
This is the first analysis. Analyze the board state and write ./actions.json with your first set of
actions.

```


Subsequent Calls

Read ./logs.txt (last log_window actions).

Recent actions and boards are at the end of the log; what changed since the last call (new moves, score transitions, plan adherence) can be informative. Check ./ for anything you saved previously, then write a new ./actions.json.

B.3.2. No-Log**First Call**

[CURRENT BOARD STATE] board

This is the first analysis. Analyze the board above and write ./actions.json with your first set of actions.

Subsequent Calls

[CURRENT BOARD STATE] Score: score | Action: action_num | Level: level Last actions: last_actions

board

What ran since the last call is listed above; check ./ for anything you saved previously, then write a new ./actions.json.

B.4. Action Space**Available Actions**

ACTION1 -- Up ACTION2 -- Down ACTION3 -- Left ACTION4 -- Right ACTION5 -- Spacebar / interact
ACTION6(x,y) -- Click at column x (0-63), row y (0-63) ACTION7 -- Undo RESET -- Reset level (actions still count)